

# Analyse des GitHub-Repos Feldtheorie

## Überblick

Das öffentliche Repository `GenesisAeon/Feldtheorie` präsentiert sich als **umfangreiche Forschungsplattform** zur Untersuchung „universeller Schwellenfelder“ (UTAC/UTF). Mit über **787 Commits, 277 Branches** und einer Fülle an Verzeichnissen (`analysis`, `models`, `data`, `scripts`, `docs`, `tests` etc.) wirkt das Projekt wie eine hybride Mischung aus Forschungspipeline, Publikationsarchiv, KI-Arbeitsumgebung und literarischer Erzählung. Laut GitHub hat das Projekt bisher **0 Stars und 0 Forks**, ist also eher ein persönliches oder internes Forschungsvorhaben als ein weit rezipiertes Open-Source-Projekt <sup>1</sup>. Das README kündigt ein konsistentes Ökosystem an, das das Feld des Schwellen-Monitoring anhand logistischer Kurven  $\sigma(\beta(R - \Theta))$  untersucht und in Version 3.0 neue Module wie die *Klimakluft  $\beta$ -Amplification*, ein *Implosive-Genesis-Framework* und eine *Sigillin-Architektur* integriert <sup>2</sup>.

## Zielsetzung

Das „Universal Threshold Field“ will untersuchen, wie abrupt auftretende Übergänge in verschiedensten Systemen (KI-Modelle, Klima, Biologie, Geophysik, Neurowissenschaften) durch die Parameter **R** (Kontrollvariable),  **$\Theta$**  (Schwelle),  **$\beta$**  (Steilheit) und  **$\zeta(R)$**  (Impedanz) beschrieben werden. Dafür werden empirische Daten kurvengefitet, Nullmodelle (linear, Potenzgesetz, Exponential) zum Vergleich herangezogen und die *Steilheit  $\beta$*  als zentrales Diagnosewerkzeug genutzt <sup>3</sup>. Die Autoren betonen, dass das Programm ein Forschungsinstrument ist und keine politischen oder sicherheitspolitischen Maßnahmen vorgibt <sup>4</sup>.

## Ordner- und Dateistruktur

- `analysis/` – Enthält zahlreiche Python-Skripte zur Datenanalyse und zum Logistik-Fitting, Batch-Konfigurationen, Ergebnisse (`analysis/results/`) sowie ein eigenes README. Hier werden z. B. `llm_beta_extractor.py` (Analyse der emergenten Fähigkeiten großer Sprachmodelle) oder `planetary_tipping_elements_fit.py` bereitgestellt. Außerdem gibt es Tools zur Metaregression, zum Bootstrapping und zum Testen multipler Hypothesen. Eine `analysis/README.md` fasst die laufenden Projekte zusammen und verweist auf aktuelle Resultate wie den **LLM-Emergent-Skill**-Datensatz, die **Amazon-Resilience-Fits** und die **Cascadia-Subduktionszone** <sup>5</sup>.
- `data/` – Sammlung von CSV-Dateien und JSON-Metadaten zu verschiedenen Domänen (AI, cognition, biology, socio\_ecology, geophysics etc.). Jedes Dataset besitzt eine begleitende `<name>.metadata.json` mit Variablenbeschreibung, Schwellenschätzungen,  $\Delta AIC$ -Werten und Lizenzangaben <sup>6</sup>.
- `models/` – Numerische Solver und Simulationen (u. a. zur Implementierung des Impedanzterms  $\zeta(R)$ ).
- `docs/` – Ausführliche Dokumentation inklusive Tri-Layer-Architektur (`docs/utac_status_alignment_v1.2.md`), Theoriedokumente (`docs/utac_theory_core.md`), API-Dokumentation, Review-Berichte und ein Release-Playbook für Zenodo <sup>7</sup>.
- `seed/` – Enthält semantische Sigillin-Dateien (Bedeutungsspeicher), Fraktaltagebuch-Einträge, Projektpläne und poetische Notizen.

- `scripts/` – Helferskripte wie `reproduce_beta.py` (schneller  $\beta$ -Fit aus CSV) oder Visualisierungswerkzeuge.
- `tests/` – Umfassende Pytest-Suite; das Projekt wirbt mit 100 % Testabdeckung in v2.0, die Badge in v3.0 zeigt aktuell etwa **29 %** <sup>8</sup>.

## Dokumentation und Governance

- **Lesbarer Einstieg** – Das README ist sehr detailliert (über 700 Zeilen) und könnte Einsteiger überfordern. Es gibt jedoch eine klare Struktur mit „Was ist neu“, „Repository-Layout“, Quick-Start-Anleitung und Daten-Governance, die die wichtigsten Elemente skizzieren.
- **Methodendefinitionen** – Die Datei `METHODS.md` erläutert das logistisches Modell, den Schätzvorgang über `scipy.optimize.curve_fit`, Initialisierungen, Randbedingungen, Nullmodelle und Bootstrapping für Unsicherheiten <sup>9</sup>. Sie definiert auch Informationskriterien (AIC, BIC), RMSE und  $R^2$  <sup>10</sup>.
- **Ethik und Limitations** – In `ETHICS.md` erklären die Autoren, dass das Projekt keine Richtlinien ersetzt, Modellunsicherheiten beachtet werden müssen und anthropomorphe Metaphern nicht wörtlich zu nehmen sind <sup>11</sup>. `LIMITATIONS.md` listet ausführlich methodische Schwächen: kleine Stichproben, Heterogenität der  $\beta$ -Werte, multiple Vergleichstests, fehlende Kausalität, Sensitivität der Preprocessing-Schritte, mangelnde externe Replikation, fehlende Cross-Validation und die bisher hypothetische Definition des Impedanzterms  $\zeta(R)$  <sup>12</sup> <sup>13</sup>. Dieses kritische Selbst-Audit wirkt vorbildlich transparent.
- **Tri-Layer-System & Sigillin** – Der Kern der Repo-Governance ist ein **Trilayer-Prinzip**: jeder „Laterne“ (Projektbestandteil) wird als YAML (Struktur), JSON (maschinell) und Markdown (Narrativ) gespeichert. Das PLAIN-Charter für Agenten beschreibt diese Regeln in verständlicher Sprache, wie man Sigillin aktualisiert, Codex-Einträge erstellt und den „Fraktaltagebuch“-Workflow einhält <sup>14</sup>. Damit wird versucht, maschinelle Konsistenz, Nachvollziehbarkeit und menschliche Lesbarkeit zu verbinden.
- **Agenten-Charter** – Das Dokument `AGENTS.md` enthält in literarischer Sprache Regeln für menschliche und KI-Agenten, die am Projekt arbeiten. Das PLAIN-Charter (`AGENTS_PLAIN.md`) übersetzt diese Regeln in einfache deutsche Sprache: alle Änderungen benötigen einen Codex-Eintrag (ähnlich einem Commit-Log), Trilayer-Synchronisation ist Pflicht und jede neue Semantik bekommt eine „Laterne“ <sup>15</sup>. Es ist ein ausgefeiltes Governance-System für Multi-Agent-Zusammenarbeit.

## Codequalität und Analyse-Skripts

Die Python-Skripte im Ordner `analysis/` erscheinen **gut strukturiert**. Sie nutzen `dataclasses`, klare Funktionsnamen und docstrings. Beispiel: `universal_beta_extractor.py` liest alle Ergebnis-JSONs, fasst Kernparameter ( $\beta$ ,  $\Theta$ ,  $\Delta AIC$ ,  $R^2$ ) zusammen und erstellt eine Master-Übersicht; die Funktionen sind defensiv programmiert und säubern Eingaben mit `_safe_float` <sup>16</sup>. `llm_beta_extractor.py` führt logistische Regression via Logit-Transformation aus, berechnet Konfidenzintervalle und vergleicht mit einem Power-Law-Nullmodell; die Implementation ist mathematisch korrekt und gut kommentiert <sup>17</sup>. In vielen Skripten werden CLI-Schnittstellen (via `argparse`) angeboten, so dass der Code auch als Kommandozeile nutzbar ist.

Einige Skripte sind sehr lang (z. B. über 1500 Zeilen) und mischen Datenmanagement, mathematische Berechnungen und Output; hier könnte eine **Modularisierung** (Utility-Funktionen in `utils/`, spezialisierte Klassen) die Wartbarkeit erhöhen. Überdies ist der Code stark an Python gebunden; funktionale Testabdeckung ist noch ausbaufähig (29 % Coverage laut Badge <sup>8</sup>).

# Daten, Forschungsergebnisse und Reproduzierbarkeit

Das Projekt legt großen Wert auf **offene Daten** und Reproduzierbarkeit:

- **Metadaten** – Für jedes Dataset existieren Metadata-Dateien; das Schema (`schemas/metadata.schema.json`) erzwingt Felder wie Variablennamen, Lizenz,  $\beta$ -Schätzung,  $\Delta$ AIC etc.
- **Reproduktions-Workflow** – Das README erklärt Schritt für Schritt, wie man mit `pip` oder `conda` eine Umgebung aufsetzt, ein Beispielskript laufen lässt (`python scripts/reproduce_beta.py --csv ...`) und wie man die Manuskript-Assets regeneriert <sup>18</sup>.
- **CI-Pipeline** – GitHub-Actions führen Linter (`ruff`, `black`), Tests und Coverage durch. Ein `codex-guard`-Workflow blockiert Pull-Requests, wenn Trilayer-Synchronisation oder Codex-Einträge fehlen (Agenten-Regelwerk) <sup>19</sup>.
- **Zenodo-Integration** – Das Repository ist mit Zenodo verknüpft (DOI 10.5281/zenodo.17472834) und enthält Playbooks zur Veröffentlichung.

## Stärken

1. **Umfassende Dokumentation** – Viele Projekte könnten sich an der Transparenz ein Beispiel nehmen: es gibt Methoden-, Ethik-, Limitations-, Test- und Governance-Dokumente. Kritische Selbstreflexion (z. B. im Limitations-Dokument) erhöht die Glaubwürdigkeit <sup>12</sup> <sup>13</sup>.
2. **Offene Daten + Reproduzierbarkeit** – Jedes Ergebnis lässt sich über Scripts und Metadaten nachrechnen; ein reproduzierbares Setup wird bereitgestellt <sup>18</sup>.
3. **Innovative Governance** – Die Tri-Layer-Sigillin-Architektur und der Codex-Mechanismus sind interessant; sie könnten als Vorlage für Multi-Agenten-Projekte dienen. Auch die PLAIN-Version erleichtert neuen Mitwirkenden den Einstieg <sup>14</sup>.
4. **Breite Anwendung** – Durch die verschiedenen Domains (AI, Klima, Biologie etc.) testet das Projekt die logistische Hypothese breit und ermöglicht Meta-Analysen.
5. **Ethik-Bewusstsein** – Risiken von Fehlinterpretationen werden offen kommuniziert; anthropomorphe Metaphern werden als Metaphern gekennzeichnet <sup>11</sup>.

## Schwächen & Verbesserungspotenzial

1. **Komplexität & Zugänglichkeit** – Das Projekt vermischt technische Analysen mit poetischer Sprache und rituellen Metaphern. Für Außenstehende ist es schwer, zwischen wissenschaftlichen Ergebnissen und stilistischen Erzählungen zu unterscheiden. Eine klare Trennung (z. B. ein rein sachliches README plus optional poetische Beilagen) könnte die Einstiegshürde senken.
2. **Überladenes README** – Das Haupt-README umfasst über 700 Zeilen und mischt Release-Notizen, theoretische Hintergründe, Beispiele und Domain-Vergleiche <sup>20</sup>. Eine Kürzung oder Auslagerung in Unterseiten würde die Lesbarkeit verbessern.
3. **Testabdeckung & Code-Modularität** – Trotz lobenswerter CI fehlt eine hohe Testabdeckung (ca. 29 %). Längere Skripte könnten modularisiert werden, um Unit-Tests zu erleichtern.
4. **Überbetonung der Universalität** – Obwohl `LIMITATIONS.md` die Heterogenität der  $\beta$ -Werte betont, wird im README immer noch von „Universeller Schwellenfeld-Model“ gesprochen. Eine klare Abgrenzung, dass  $\beta$  in der Tendenz *clustert*, aber **kein universeller Naturkonstant** ist, wäre wichtig <sup>21</sup>.
5. **Fehlender Community-Bezug** – Bei 0 Stars und 0 Forks wirkt das Projekt eher wie ein privates Labor, obwohl es sich an der Open-Science-Community orientieren möchte. Das Recruiting von Peer-Reviewern oder das Erstellen eines einfachen Beispielprojekts könnte externe Mitwirkende anziehen.

6. **Verbesserungswürdige Code-Kommentare** – Einige Skripte enthalten poetische Beschreibungen, die die technische Funktion verschleiern. Standard-Kommentare, Docstrings und eine saubere Einhaltung von Python-PEP-8-Konventionen wären hilfreich.
7. **Daten-Sicherheit** – Die Governance-Dokumente erinnern daran, personenbezogene Daten zu anonymisieren. Dennoch sollte überprüft werden, ob in `data/` sensible Daten gespeichert werden.

## Gesamtfazit

`Feldtheorie` ist kein gewöhnliches Software-Repo, sondern eine **experimentelle Forschungsumgebung** mit ungewöhnlichem Storytelling. Die Arbeit investiert viel in **offene Dokumentation, Reproduzierbarkeit und ethische Reflexion**. Die Logistik-Analyse-Skripte sind sauber implementiert und nutzen moderne Python-Technik wie `dataclasses` <sup>22</sup>. Gleichzeitig erschweren die poetische Sprache, die enorme Dateimenge und die komplexe Governance den Einstieg. Wer tiefer einsteigen möchte, sollte sich zunächst die PLAIN-Agenten-Regeln und das Methoden-Dokument ansehen und anhand eines Beispiel-Datasets (`wei_emergent_abilities.csv`) die Pipeline nachvollziehen. Eine klarere Trennung zwischen **wissenschaftlicher Analyse** und **metaphorischem Manifest** könnte helfen, das Projekt einer breiteren Community zugänglich zu machen.

---

<sup>1</sup> GitHub - GenesisAeon/Feldtheorie

<https://github.com/GenesisAeon/Feldtheorie>

<sup>2</sup> <sup>3</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>18</sup> <sup>20</sup> Feldtheorie/README.md at main · GenesisAeon/Feldtheorie · GitHub

<https://github.com/GenesisAeon/Feldtheorie/blob/main/README.md>

<sup>4</sup> <sup>11</sup> Feldtheorie/ETHICS.md at main · GenesisAeon/Feldtheorie · GitHub

<https://github.com/GenesisAeon/Feldtheorie/blob/main/ETHICS.md>

<sup>5</sup> Feldtheorie/analysis at main · GenesisAeon/Feldtheorie · GitHub

<https://github.com/GenesisAeon/Feldtheorie/tree/main/analysis>

<sup>9</sup> <sup>10</sup> Feldtheorie/METHODS.md at main · GenesisAeon/Feldtheorie · GitHub

<https://github.com/GenesisAeon/Feldtheorie/blob/main/METHODS.md>

<sup>12</sup> <sup>13</sup> <sup>21</sup> Feldtheorie/LIMITATIONS.md at main · GenesisAeon/Feldtheorie · GitHub

<https://github.com/GenesisAeon/Feldtheorie/blob/main/LIMITATIONS.md>

<sup>14</sup> <sup>15</sup> <sup>19</sup> Feldtheorie/AGENTS\_PLAIN.md at main · GenesisAeon/Feldtheorie · GitHub

[https://github.com/GenesisAeon/Feldtheorie/blob/main/AGENTS\\_PLAIN.md](https://github.com/GenesisAeon/Feldtheorie/blob/main/AGENTS_PLAIN.md)

<sup>16</sup> <sup>22</sup> [raw.githubusercontent.com](https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/universal_beta_extractor.py)

[https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/universal\\_beta\\_extractor.py](https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/universal_beta_extractor.py)

<sup>17</sup> [raw.githubusercontent.com](https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/llm_beta_extractor.py)

[https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/llm\\_beta\\_extractor.py](https://raw.githubusercontent.com/GenesisAeon/Feldtheorie/main/analysis/llm_beta_extractor.py)